knitweb-lens: Pip Promotion Strategy & Tactics

Package: `knitweb-lens`
Target PyPI URL: `https://pypi.org/project/knitweb-lens`
Repo: `https://github.com/Knitweb/lens`
Date: 2026-06-30

Executive Summary

`knitweb-lens` is the pip entry point for the Knitweb P2P quantum circuit network. The strategy is **Colab-first, search-visible, referral-driven**: make the library frictionless to discover in Google Colab, rank on PyPI and Google for "quantum circuit library python", and convert quantum researchers into relay contributors.

Part 1 — Product Strategy

1.1 Why a standalone pip package?

The knitweb ecosystem is powerful but opaque to outsiders. `knitweb-lens` is the smallest useful unit: install one package, get 100 working quantum circuits, store your own, share them. No Knitweb account required. The P2P layer is opt-in.

1.2 Positioning

Segment	How knitweb-lens wins
Students (Qiskit/PennyLane tutorial users)	Ready-to-run reference circuits, no setup friction
Researchers	CID-based reproducibility: cite a CID, share a CID
Quantum cloud users (IBM/AWS Braket)	QASM output works with any cloud backend
Cryptography researchers	secp256k1 oracle, ECDLP, BB84/E91 built-in
P2P / decentralisation enthusiasts	Content-addressed sharing without a server

1.3 Distribution Tiers

```
Tier 0 — Zero-install:   Colab "Open in Colab" badge (no local install)
Tier 1 — One-liner:      pip install knitweb-lens
Tier 2 — Full stack:     pip install knitweb-lens[all] + lens relay join
Tier 3 — Node operator:  Run a Knitweb relay, earn PLS token incentives
```

Part 2 — Promotion Tactics

2.1 PyPI & Search Optimisation

pyproject.toml keywords (already set):

```
keywords = ["quantum", "circuits", "qasm", "p2p", "knitweb",
            "qiskit", "pennylane", "cirq", "grover", "vqe", "qaoa"]
```

README anchors that Google/PyPI will index: - "quantum circuit library python" - "qasm 2.0 circuit collection" - "p2p quantum circuits" - "content-addressed quantum circuit"

PyPI long description: The full README is the PyPI page. Include the circuit domain table, adapter matrix, and Colab badge above the fold.

2.2 Colab Notebooks

Four notebooks, each with an "Open in Colab" badge. Publish to `github.com/Knitweb/lens/notebooks/`.

Distribution path: 1. Pin in GitHub repo README 2. Share on `r/QuantumComputing` and `r/qiskit` when package hits 1.0 3. Submit to Qiskit Ecosystem as "circuit library" 4. Post to PennyLane Community

2.3 GitHub Strategy

Repository checklist: - [ ] `github.com/Knitweb/lens` — main repo - [ ] Topics: `quantum`, `qasm`, `qiskit`, `p2p`, `knitweb`, `circuits` - [ ] Releases: tag `v0.1.0` on first PyPI publish - [ ] GitHub Pages: generate circuit browser from library specs

Issue-driven roadmap (public): | Milestone | Target | |---|---| | v0.1 — 100 circuits + adapters | Now | | v0.2 — Relay-backed P2P | +4 weeks | | v0.3 — Gither search integration | +8 weeks | | v0.4 — PLS token rewards for circuit submission | +16 weeks | | v1.0 — 500 circuits + Cirq/tket full support | +6 months |

2.4 Community Seeding

Phase 1 (week 1-2): Quiet launch - Publish to PyPI - Post in Knitweb Discord / Telegram - DM 5 active Qiskit contributors: "can you test this adapter?"

Phase 2 (week 3-4): Community posts - `r/QuantumComputing`: "We built a P2P quantum circuit library — 100 circuits, QASM, pip install" - Hacker News: Show HN (title: "knitweb-lens: pip install quantum circuits, share by CID") - Twitter/X thread: 5 tweets with one circuit visualisation each day

Phase 3 (month 2): Conference & education - Submit poster/demo to IEEE Quantum Week 2026 - Contact university quantum computing courses: "free circuit library for students" - Approach Quantum

Open Source Foundation (QOSF) for listing

Phase 4 (month 3+): Token incentives - Launch circuit bounties: submit 10 circuits → earn PLS tokens - Leaderboard on `gi.ther.io/circuits` - Partner with quantum cloud providers (IBM, IonQ, Quantinuum) for featured circuits

2.5 Content Marketing

Blog posts (publish on Medium + dev.to): 1. "100 quantum circuits in one pip install" — overview 2. "How we use content addressing (CIDs) for reproducible quantum research" 3. "Quantum circuits on the P2P web: why QASM + SHA-256 is the right interface" 4. "From Qiskit to Knitweb in 3 lines of code"

Circuit of the week (Twitter/GitHub Discussions): Each week, highlight one circuit with: what it does, QASM source, how to run it, why it matters. Auto-generate from the library spec.

Part 3 — Knitweb / Gither / Intelfield Integration

3.1 Knitweb Fabric Indexing

Every stored circuit creates a warp/weft entry in the Knitweb fabric:

```
Warp (entity):    circuit:<name>
Weft (assertion): cid → <lcid:…>
                  qubits → <n>
                  domain → <domain>
                  tags → [tag1, tag2]
```

This makes circuits queryable via any Knitweb node's `interpret` gateway (PR #157).

3.2 Gither Registry

Gither (`gi.ther.io`) is the decentralised code/artefact registry. Each circuit is published as a Gither artefact:

```
# After publishing to PyPI:
gither publish knitweb-lens:bell_phi_plus --cid lcid:a3f9...
```

Users can then `gither fetch circuit:bell_phi_plus` — no PyPI required.

3.3 Intelfield (GitHub Project #6)

Circuit library development is tracked in Intelfield. Issues #310 and #311 (from the WNW backlog) cover: - **#310** Quantum circuit bridge — lens → Knitweb fabric weft - **#311** Quantum Forge runs — automated circuit validation CI

Add `knitweb-lens` release tasks to project #6 for visibility.

Part 4 — Code: Key Snippets

4.1 Quick install and use

```
# Install
# pip install knitweb-lens

from knitweb_lens import library, search
from knitweb_lens.store import Store

# Get all 100 circuits
lib = library()
print(len(lib))  # 100

# Find by keyword
results = search('grover', domain='algorithms')

# Get QASM
qasm = lib['bell_phi_plus'].qasm

# Store and get CID
store = Store()
cid = store.put(lib['ghz_5'])
print(cid)  # lcid:…
```

4.2 Convert Qiskit circuit

```
from qiskit import QuantumCircuit
from knitweb_lens.adapters import from_qiskit
from knitweb_lens.store import Store

qc = QuantumCircuit(4)
qc.h(range(4))
qc.cx(0, 1); qc.cx(1, 2); qc.cx(2, 3)
qc.measure_all()

circ = from_qiskit(qc, name='ghz4_qiskit', domain='fundamental')
store = Store(relay='https://relay.5mart.ml')
cid = store.put(circ)
print(f'Published: {cid}')
```

4.3 CLI workflow

```
# List all cryptography circuits
lens list --domain cryptography

# Show Shor's circuit QASM
lens show shor_order_15

# Push your own
lens push ./my_algorithm.qasm --name my_algo --domain algorithms

# Search and pull
lens search vqe --domain optimization
lens pull lcid:c8a21f... --out retrieved.qasm
```

4.4 Knitweb fabric bridge (post v0.2)

```
from knitweb_lens import library
from knitweb_lens.store import Store
# Future: from knitweb.lens import KnitwebLensAdapter

store = Store(relay='https://relay.5mart.ml')
lib = library()

# Publish all 100 built-in circuits to the relay
for name, circ in lib.items():
    cid = store.put(circ)
    # Each CID is automatically discoverable on the Knitweb fabric
    print(f'{name} → {cid}')
```

Part 5 — Success Metrics

Metric	1 month	3 months	12 months
PyPI downloads/week	100	500	5,000
GitHub stars	50	200	1,000
Circuits in network	100	500	5,000
Relay nodes	1	5	50
Community contributors	2	10	50
Colab notebook opens	200	1,000	10,000

Appendix A — PyPI Publishing Commands

```
# One-time setup
pip install build twine

# Build
cd /tmp/lens-pkg
python -m build

# Upload to TestPyPI first
twine upload --repository testpypi dist/*

# Upload to PyPI
twine upload dist/*
```

Environment variables needed:

```
TWINE_USERNAME = __token__
TWINE_PASSWORD = <your PyPI API token>
```

Appendix B — GitHub Actions CI

File: `.github/workflows/publish.yml`

```
name: Publish to PyPI
on:
  push:
    tags: ['v*']
jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: pip install build twine
      - run: python -m build
      - run: twine upload dist/*
        env:
          TWINE_USERNAME: __token__
          TWINE_PASSWORD: ${{ secrets.PYPI_TOKEN }}
```

Appendix C — Relay Server (v0.2)

The Knitweb relay is a thin FastAPI server that stores circuits as JSON blobs keyed by CID. It reuses the existing Knitweb P2P bridge at `relay.5mart.ml`.

```
# Minimal relay server
from fastapi import FastAPI, HTTPException
from pathlib import Path
import json

app = FastAPI()
STORE = Path('/var/knitweb_lens')
STORE.mkdir(exist_ok=True)

@app.put('/circuits/{cid}')
async def put_circuit(cid: str, data: dict):
    (STORE / f'{cid}.json').write_text(json.dumps(data))
    return {'ok': True}

@app.get('/circuits/{cid}')
async def get_circuit(cid: str):
    p = STORE / f'{cid}.json'
    if not p.exists():
        raise HTTPException(404)
    return json.loads(p.read_text())
```